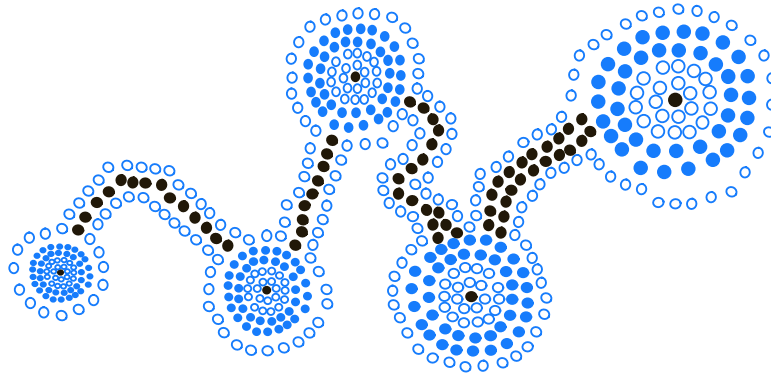




Contents

12.1 Merge-Sort	532
12.1.1 Divide-and-Conquer	532
12.1.2 Array-Based Implementation of Merge-Sort	537
12.1.3 The Running Time of Merge-Sort	538
12.1.4 Merge-Sort and Recurrence Equations ★	540
12.1.5 Alternative Implementations of Merge-Sort	541
12.2 Quick-Sort	544
12.2.1 Randomized Quick-Sort	551
12.2.2 Additional Optimizations for Quick-Sort	553
12.3 Studying Sorting through an Algorithmic Lens	556
12.3.1 Lower Bound for Sorting	556
12.3.2 Linear-Time Sorting: Bucket-Sort and Radix-Sort	558
12.4 Comparing Sorting Algorithms	561
12.5 Selection	563
12.5.1 Prune-and-Search	563
12.5.2 Randomized Quick-Select	564
12.5.3 Analyzing Randomized Quick-Select	565
12.6 Exercises	566

12.1 Merge-Sort

We have introduced several sorting algorithms thus far, including insertion-sort (see Sections 3.1.2, 7.6, and 9.4.1); selection-sort (see Section 9.4.1); bubble-sort (see Exercise C-7.51); and heap-sort (see Section 9.4.2). In this chapter, we will present four other sorting algorithms, called *merge-sort*, *quick-sort*, *bucket-sort*, and *radix-sort*, and then discuss the advantages and disadvantages of the various algorithms in Section 12.4.

12.1.1 Divide-and-Conquer

The first two algorithms we describe in this chapter, merge-sort and quick-sort, use recursion in an algorithmic design pattern called *divide-and-conquer*. We have already seen the power of recursion in describing algorithms in an elegant manner (see Chapter 5). The divide-and-conquer pattern consists of the following three steps:

1. **Divide:** If the input size is smaller than a certain threshold (say, one or two elements), solve the problem directly using a straightforward method and return the solution so obtained. Otherwise, divide the input data into two or more disjoint subsets.
2. **Conquer:** Recursively solve the subproblems associated with the subsets.
3. **Combine:** Take the solutions to the subproblems and merge them into a solution to the original problem.

Using Divide-and-Conquer for Sorting

We first describe the merge-sort algorithm at a high level, without focusing on whether the data is an array or linked list. (We will soon give concrete implementations for each.) To sort a sequence S with n elements using the three divide-and-conquer steps, the merge-sort algorithm proceeds as follows:

1. **Divide:** If S has zero or one element, return S immediately; it is already sorted. Otherwise (S has at least two elements), remove all the elements from S and put them into two sequences, S_1 and S_2 , each containing about half of the elements of S ; that is, S_1 contains the first $\lfloor n/2 \rfloor$ elements of S , and S_2 contains the remaining $\lceil n/2 \rceil$ elements.
2. **Conquer:** Recursively sort sequences S_1 and S_2 .
3. **Combine:** Put the elements back into S by merging the sorted sequences S_1 and S_2 into a sorted sequence.

In reference to the divide step, we recall that the notation $\lfloor x \rfloor$ indicates the *floor* of x , that is, the largest integer k , such that $k \leq x$. Similarly, the notation $\lceil x \rceil$ indicates the *ceiling* of x , that is, the smallest integer m , such that $x \leq m$.

We can visualize an execution of the merge-sort algorithm by means of a binary tree T , called the **merge-sort tree**. Each node of T represents a recursive invocation (or call) of the merge-sort algorithm. We associate with each node v of T the sequence S that is processed by the invocation associated with v . The children of node v are associated with the recursive calls that process the subsequences S_1 and S_2 of S . The external nodes of T are associated with individual elements of S , corresponding to instances of the algorithm that make no recursive calls.

Figure 12.1 summarizes an execution of the merge-sort algorithm by showing the input and output sequences processed at each node of the merge-sort tree. The step-by-step evolution of the merge-sort tree is shown in Figures 12.2 through 12.4.

This algorithm visualization in terms of the merge-sort tree helps us analyze the running time of the merge-sort algorithm. In particular, since the size of the input sequence roughly halves at each recursive call of merge-sort, the height of the merge-sort tree is about $\log n$ (recall that the base of log is 2 if omitted).

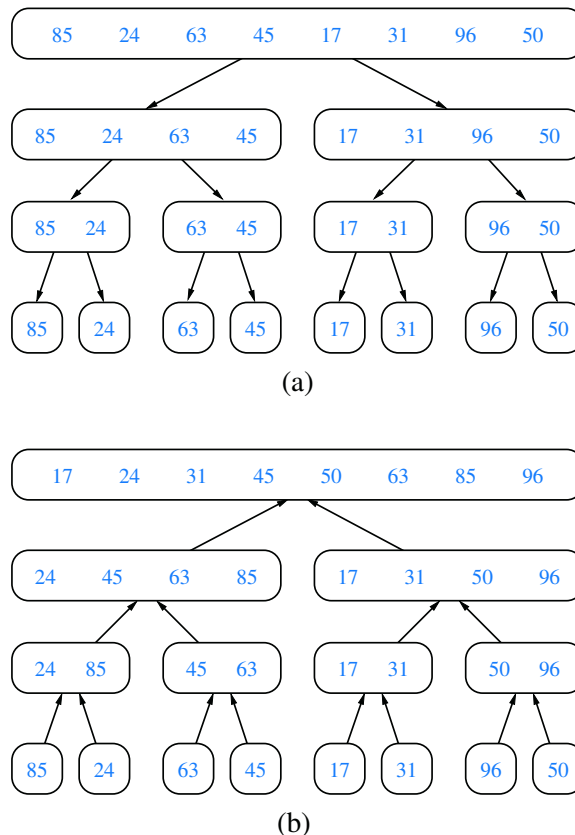


Figure 12.1: Merge-sort tree T for an execution of the merge-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T ; (b) output sequences generated at each node of T .

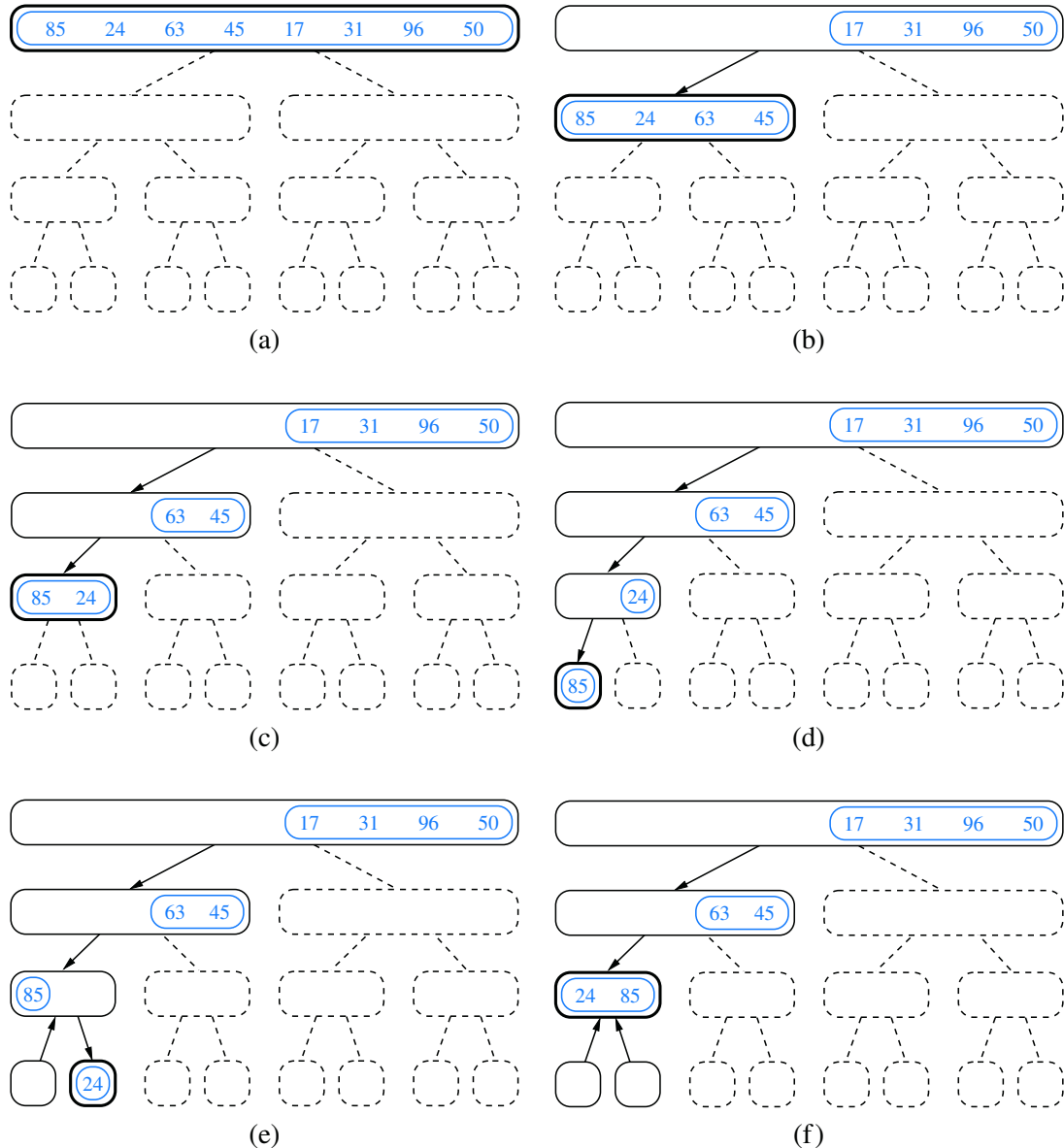


Figure 12.2: Visualization of an execution of merge-sort. Each node of the tree represents a recursive call of merge-sort. The nodes drawn with dashed lines represent calls that have not been made yet. The node drawn with thick lines represents the current call. The empty nodes drawn with thin lines represent completed calls. The remaining nodes (drawn with thin lines and not empty) represent calls that are waiting for a child call to return. (Continues in Figure 12.3.)

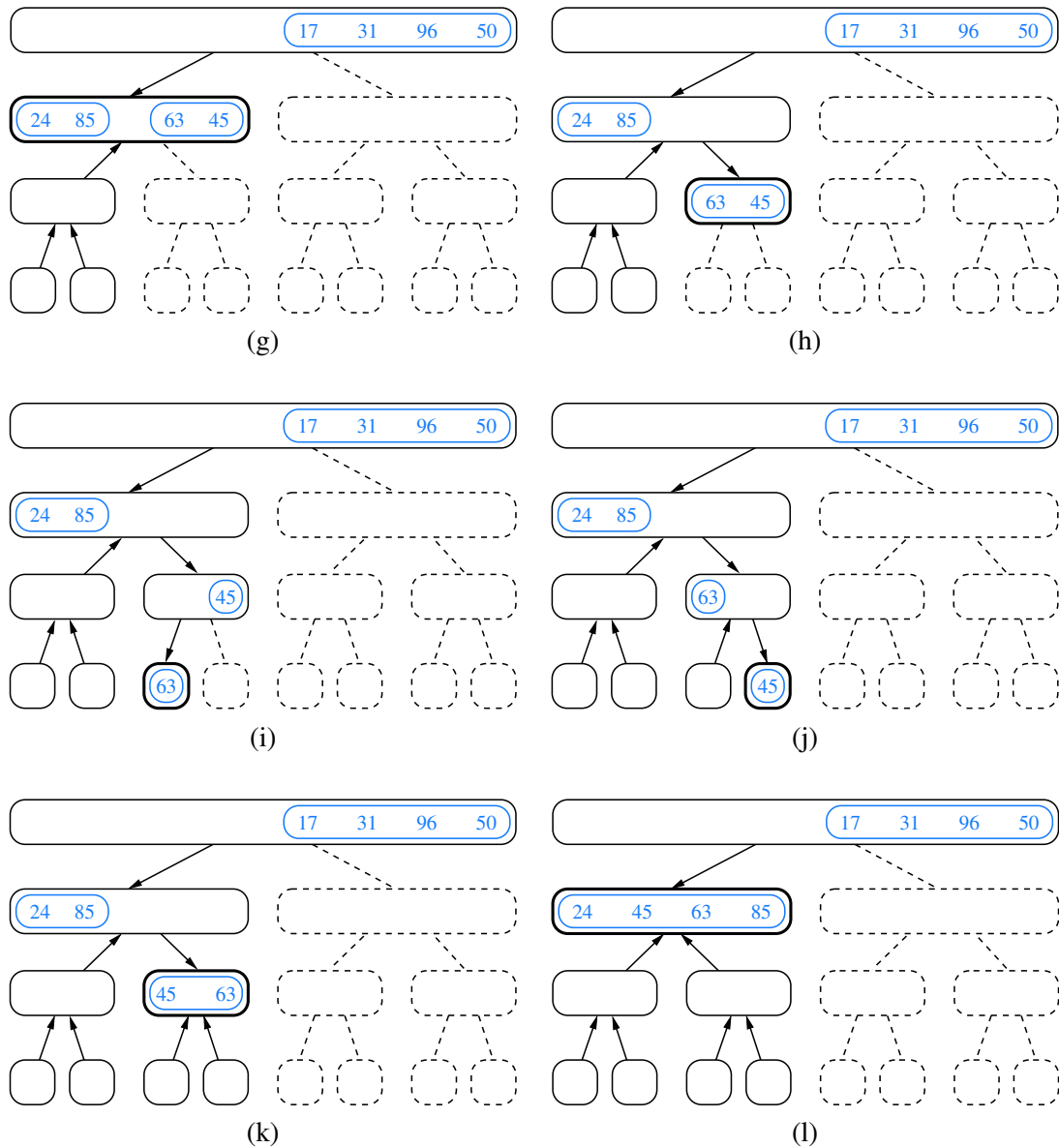


Figure 12.3: Visualization of an execution of merge-sort. (Combined with Figures 12.2 and 12.4.)

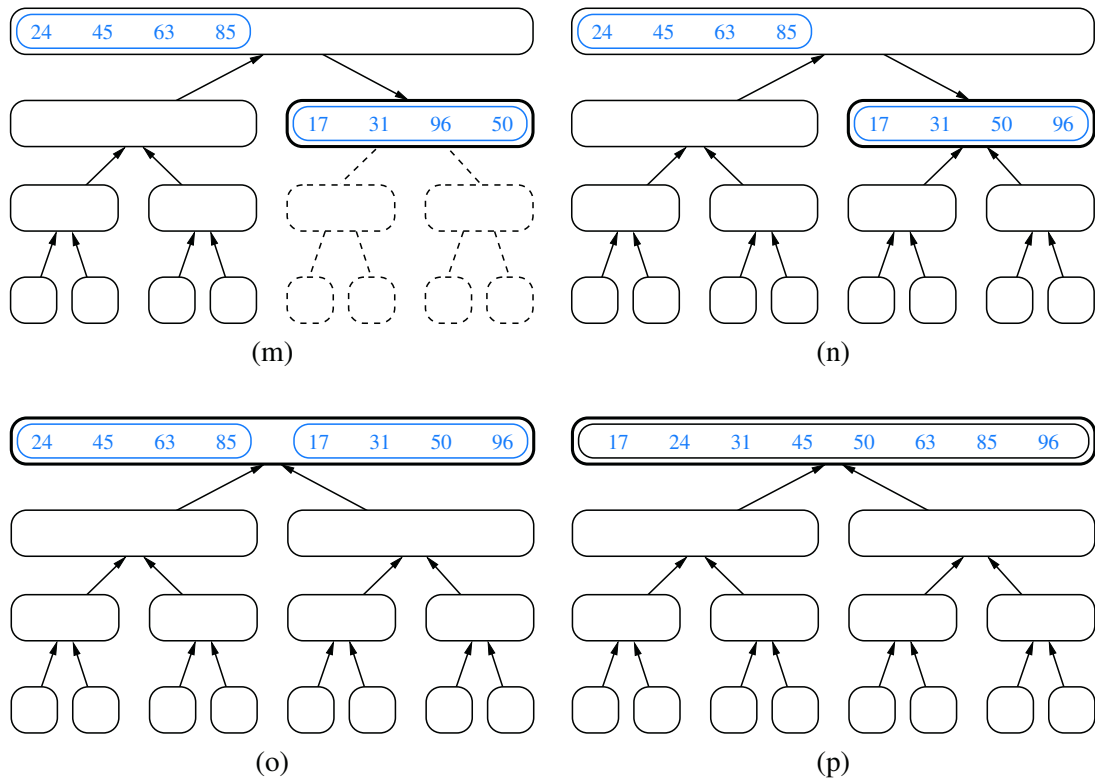


Figure 12.4: Visualization of an execution of merge-sort (continued from Figure 12.3). Several calls are omitted between (m) and (n). Note the merging of two halves performed in step (p).

Proposition 12.1: *The merge-sort tree associated with an execution of merge-sort on a sequence of size n has height $\lceil \log n \rceil$.*

We leave the justification of Proposition 12.1 as a simple exercise (R-12.1). We will use this proposition to analyze the running time of the merge-sort algorithm.

Having given an overview of merge-sort and an illustration of how it works, let us consider each of the steps of this divide-and-conquer algorithm in more detail. Dividing a sequence of size n involves separating it at the element with index $\lceil n/2 \rceil$, and recursive calls can be started by passing these smaller sequences as parameters. The difficult step is combining the two sorted sequences into a single sorted sequence. Thus, before we present our analysis of merge-sort, we need to say more about how this is done.

12.1.2 Array-Based Implementation of Merge-Sort

We begin by focusing on the case when a sequence of items is represented with an array. The merge method (Code Fragment 12.1) is responsible for the subtask of merging two previously sorted sequences, S_1 and S_2 , with the output copied into S . We copy one element during each pass of the while loop, conditionally determining whether the next element should be taken from S_1 or S_2 . The divide-and-conquer merge-sort algorithm is given in Code Fragment 12.2.

We illustrate a step of the merge process in Figure 12.5. During the process, index i represents the number of elements of S_1 that have been copied to S , while index j represents the number of elements of S_2 that have been copied to S . Assuming S_1 and S_2 both have at least one uncopied element, we copy the smaller of the two elements being considered. Since $i + j$ objects have been previously copied, the next element is placed in $S[i + j]$. (For example, when $i + j$ is 0, the next element is copied to $S[0]$). If we reach the end of one of the sequences, we must copy the next element from the other.

```

1  /** Merge contents of arrays S1 and S2 into properly sized array S. */
2  public static <K> void merge(K[] S1, K[] S2, K[] S, Comparator<K> comp) {
3      int i = 0, j = 0;
4      while (i + j < S.length) {
5          if (j == S2.length || (i < S1.length && comp.compare(S1[i], S2[j]) < 0))
6              S[i+j] = S1[i++];           // copy ith element of S1 and increment i
7          else
8              S[i+j] = S2[j++];           // copy jth element of S2 and increment j
9      }
10 }
```

Code Fragment 12.1: An implementation of the merge operation for a Java array.

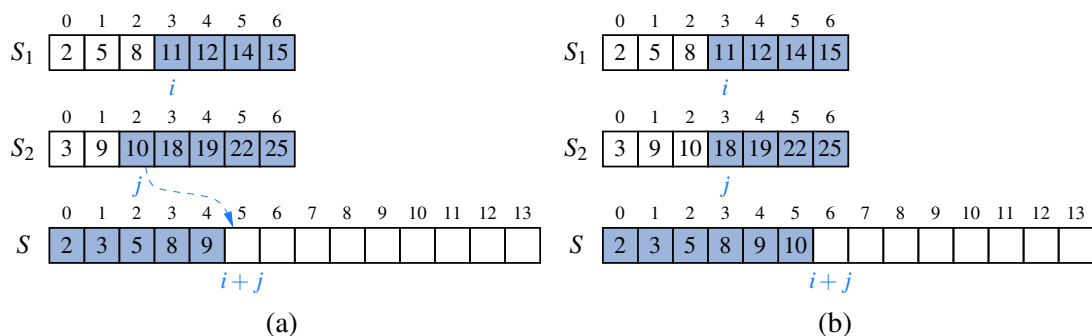


Figure 12.5: A step in the merge of two sorted arrays for which $S_2[j] < S_1[i]$. We show the arrays before the copy step in (a) and after it in (b).

```

1  /** Merge-sort contents of array S. */
2  public static <K> void mergeSort(K[] S, Comparator<K> comp) {
3      int n = S.length;
4      if (n < 2) return;                                // array is trivially sorted
5      // divide
6      int mid = n/2;
7      K[] S1 = Arrays.copyOfRange(S, 0, mid);           // copy of first half
8      K[] S2 = Arrays.copyOfRange(S, mid, n);           // copy of second half
9      // conquer (with recursion)
10     mergeSort(S1, comp);                               // sort copy of first half
11     mergeSort(S2, comp);                               // sort copy of second half
12     // merge results
13     merge(S1, S2, S, comp);                           // merge sorted halves back into original
14 }

```

Code Fragment 12.2: An implementation of the recursive merge-sort algorithm for a Java array (using the merge method defined in Code Fragment 12.1).

We note that methods `merge` and `mergeSort` rely on use of a `Comparator` instance to compare a pair of generic objects that are presumed to belong to a total order. This is the same approach we introduced when defining priority queues in Section 9.2.2, and when studying implementing sorted maps in Chapters 10 and 11.

12.1.3 The Running Time of Merge-Sort

We begin by analyzing the running time of the merge algorithm. Let n_1 and n_2 be the number of elements of S_1 and S_2 , respectively. It is clear that the operations performed inside each pass of the while loop take $O(1)$ time. The key observation is that during each iteration of the loop, one element is copied from either S_1 or S_2 into S (and that element is considered no further). Therefore, the number of iterations of the loop is $n_1 + n_2$. Thus, the running time of algorithm `merge` is $O(n_1 + n_2)$.

Having analyzed the running time of the merge algorithm used to combine subproblems, let us analyze the running time of the entire merge-sort algorithm, assuming it is given an input sequence of n elements. For simplicity, we restrict our attention to the case where n is a power of 2. We leave it to an exercise (R-12.3) to show that the result of our analysis also holds when n is not a power of 2.

When evaluating the merge-sort recursion, we rely on the analysis technique introduced in Section 5.2. We account for the amount of time spent within each recursive call, but excluding any time spent waiting for successive recursive calls to terminate. In the case of our `mergeSort` method, we account for the time to divide the sequence into two subsequences, and the call to `merge` to combine the two sorted sequences, but we exclude the two recursive calls to `mergeSort`.

A merge-sort tree T , as portrayed in Figures 12.2 through 12.4, can guide our analysis. Consider a recursive call associated with a node v of the merge-sort tree T . The divide step at node v is straightforward; this step runs in time proportional to the size of the sequence for v , based on the use of slicing to create copies of the two list halves. We have already observed that the merging step also takes time that is linear in the size of the merged sequence. If we let i denote the depth of node v , the time spent at node v is $O(n/2^i)$, since the size of the sequence handled by the recursive call associated with v is equal to $n/2^i$.

Looking at the tree T more globally, as shown in Figure 12.6, we see that, given our definition of “time spent at a node,” the running time of merge-sort is equal to the sum of the times spent at the nodes of T . Observe that T has exactly 2^i nodes at depth i . This simple observation has an important consequence, for it implies that the overall time spent at all the nodes of T at depth i is $O(2^i \cdot n/2^i)$, which is $O(n)$. By Proposition 12.1, the height of T is $\lceil \log n \rceil$. Thus, since the time spent at each of the $\lceil \log n \rceil + 1$ levels of T is $O(n)$, we have the following result:

Proposition 12.2: *Algorithm merge-sort sorts a sequence S of size n in $O(n \log n)$ time, assuming two elements of S can be compared in $O(1)$ time.*

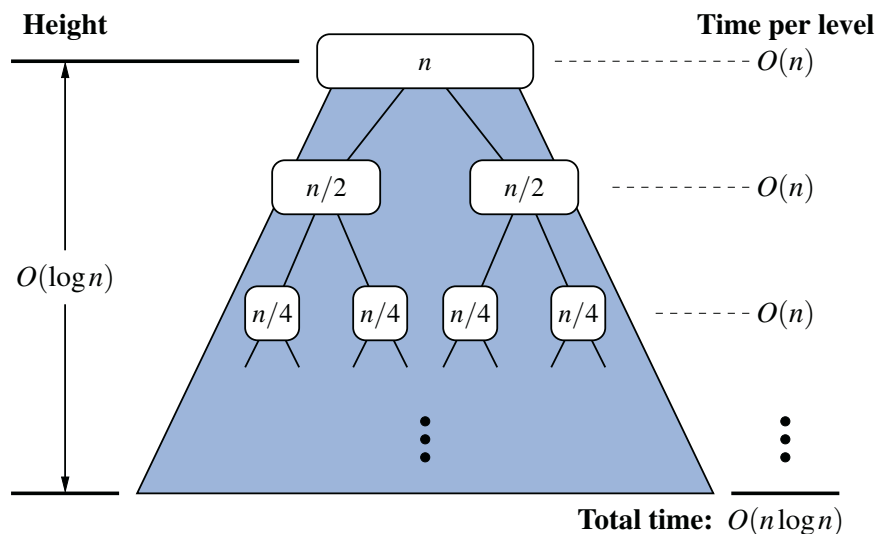


Figure 12.6: A visual analysis of the running time of merge-sort. Each node represents the time spent in a particular recursive call, labeled with the size of its subproblem.

12.1.4 Merge-Sort and Recurrence Equations ★

There is another way to justify that the running time of the merge-sort algorithm is $O(n \log n)$ (Proposition 12.2). Namely, we can deal more directly with the recursive nature of the merge-sort algorithm. In this section, we will present such an analysis of the running time of merge-sort, and in so doing, introduce the mathematical concept of a **recurrence equation** (also known as **recurrence relation**).

Let the function $t(n)$ denote the worst-case running time of merge-sort on an input sequence of size n . Since merge-sort is recursive, we can characterize function $t(n)$ by means of an equation where the function $t(n)$ is recursively expressed in terms of itself. In order to simplify our characterization of $t(n)$, let us restrict our attention to the case when n is a power of 2. (We leave the problem of showing that our asymptotic characterization still holds in the general case as an exercise.) In this case, we can specify the definition of $t(n)$ as

$$t(n) = \begin{cases} b & \text{if } n \leq 1 \\ 2t(n/2) + cn & \text{otherwise.} \end{cases}$$

An expression such as the one above is called a recurrence equation, since the function appears on both the left- and right-hand sides of the equal sign. Although such a characterization is correct and accurate, what we really desire is a big-Oh type of characterization of $t(n)$ that does not involve the function $t(n)$ itself. That is, we want a **closed-form** characterization of $t(n)$.

We can obtain a closed-form solution by applying the definition of a recurrence equation, assuming n is relatively large. For example, after one more application of the equation above, we can write a new recurrence for $t(n)$ as

$$\begin{aligned} t(n) &= 2(2t(n/2^2) + (cn/2)) + cn \\ &= 2^2t(n/2^2) + 2(cn/2) + cn = 2^2t(n/2^2) + 2cn. \end{aligned}$$

If we apply the equation again, we get $t(n) = 2^3t(n/2^3) + 3cn$. At this point, we should see a pattern emerging, so that after applying this equation i times, we get

$$t(n) = 2^i t(n/2^i) + icn.$$

The issue that remains, then, is to determine when to stop this process. To see when to stop, recall that we switch to the closed form $t(n) = b$ when $n \leq 1$, which will occur when $2^i = n$. In other words, this will occur when $i = \log n$. Making this substitution, then, yields

$$\begin{aligned} t(n) &= 2^{\log n} t(n/2^{\log n}) + (\log n)cn \\ &= nt(1) + cn \log n \\ &= nb + cn \log n. \end{aligned}$$

That is, we get an alternative justification of the fact that $t(n)$ is $O(n \log n)$.

12.1.5 Alternative Implementations of Merge-Sort

Sorting Linked Lists

The merge-sort algorithm can easily be adapted to use any form of a basic queue as its container type. In Code Fragment 12.3, we provide such an implementation, based on use of the `LinkedList` class from Section 6.2.3. The $O(n \log n)$ bound for merge-sort from Proposition 12.2 applies to this implementation as well, since each basic operation runs in $O(1)$ time when implemented with a linked list. We show an example execution of this version of the merge algorithm in Figure 12.7.

```

1  /** Merge contents of sorted queues S1 and S2 into empty queue S. */
2  public static <K> void merge(Queue<K> S1, Queue<K> S2, Queue<K> S,
3                               Comparator<K> comp) {
4      while (!S1.isEmpty() && !S2.isEmpty()) {
5          if (comp.compare(S1.first(), S2.first()) < 0)
6              S.enqueue(S1.dequeue());           // take next element from S1
7          else
8              S.enqueue(S2.dequeue());           // take next element from S2
9      }
10     while (!S1.isEmpty())
11         S.enqueue(S1.dequeue());               // move any elements that remain in S1
12     while (!S2.isEmpty())
13         S.enqueue(S2.dequeue());               // move any elements that remain in S2
14 }
15
16 /** Merge-sort contents of queue. */
17 public static <K> void mergeSort(Queue<K> S, Comparator<K> comp) {
18     int n = S.size();
19     if (n < 2) return;                          // queue is trivially sorted
20     // divide
21     Queue<K> S1 = new LinkedList<>();           // (or any queue implementation)
22     Queue<K> S2 = new LinkedList<>();
23     while (S1.size() < n/2)
24         S1.enqueue(S.dequeue());               // move the first n/2 elements to S1
25     while (!S.isEmpty())
26         S2.enqueue(S.dequeue());               // move remaining elements to S2
27     // conquer (with recursion)
28     mergeSort(S1, comp);                       // sort first half
29     mergeSort(S2, comp);                       // sort second half
30     // merge results
31     merge(S1, S2, S, comp);                   // merge sorted halves back into original
32 }

```

Code Fragment 12.3: An implementation of merge-sort using a basic queue.

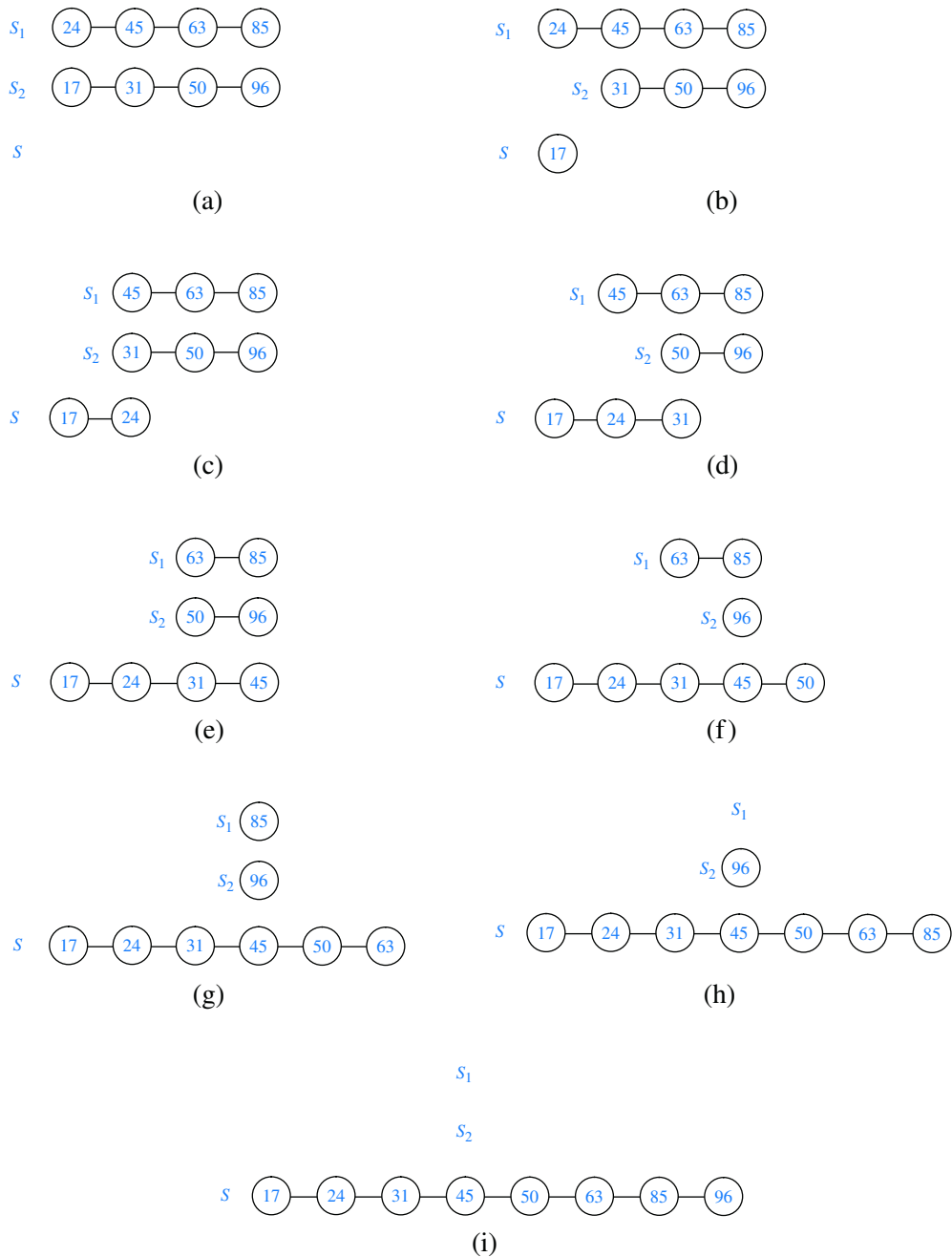


Figure 12.7: Example of an execution of the merge algorithm, as implemented in Code Fragment 12.3 using queues.

A Bottom-Up (Nonrecursive) Merge-Sort

There is a nonrecursive version of array-based merge-sort, which runs in $O(n \log n)$ time. It is a bit faster than recursive merge-sort in practice, as it avoids the extra overheads of recursive calls and temporary memory at each level. The main idea is to perform merge-sort bottom-up, performing the merges level by level going up the merge-sort tree. Given an input array of elements, we begin by merging every successive pair of elements into sorted runs of length two. We merge these runs into runs of length four, merge these new runs into runs of length eight, and so on, until the array is sorted. To keep the space usage reasonable, we deploy a second array that stores the merged runs (swapping input and output arrays after each iteration). We give a Java implementation in Code Fragment 12.4, using the built-in method `System.arraycopy` to copy a range of cells between two arrays. A similar bottom-up approach can be used for sorting linked lists. (See Exercise C-12.30.)

```

1  /** Merges in[start..start+inc-1] and in[start+inc..start+2*inc-1] into out. */
2  public static <K> void merge(K[] in, K[] out, Comparator<K> comp,
3                                int start, int inc) {
4      int end1 = Math.min(start + inc, in.length);           // boundary for run 1
5      int end2 = Math.min(start + 2 * inc, in.length);       // boundary for run 2
6      int x=start;                                           // index into run 1
7      int y=start+inc;                                       // index into run 2
8      int z=start;                                           // index into output
9      while (x < end1 && y < end2)
10         if (comp.compare(in[x], in[y]) < 0)
11             out[z++] = in[x++];                             // take next from run 1
12         else
13             out[z++] = in[y++];                             // take next from run 2
14     if (x < end1) System.arraycopy(in, x, out, z, end1 - x); // copy rest of run 1
15     else if (y < end2) System.arraycopy(in, y, out, z, end2 - y); // copy rest of run 2
16 }
17 /** Merge-sort contents of data array. */
18 public static <K> void mergeSortBottomUp(K[] orig, Comparator<K> comp) {
19     int n = orig.length;
20     K[] src = orig;                                         // alias for the original
21     K[] dest = (K[]) new Object[n];                       // make a new temporary array
22     K[] temp;                                              // reference used only for swapping
23     for (int i=1; i < n; i *= 2) {                          // each iteration sorts all runs of length i
24         for (int j=0; j < n; j += 2*i)                     // each pass merges two runs of length i
25             merge(src, dest, comp, j, i);
26         temp = src; src = dest; dest = temp;              // reverse roles of the arrays
27     }
28     if (orig != src)
29         System.arraycopy(src, 0, orig, 0, n);             // additional copy to get result to original
30 }

```

Code Fragment 12.4: An implementation of the nonrecursive merge-sort algorithm.